KUTAISI INTERNATIONAL UNIVERSITY

Final Project Report

GeoArchive

A Laravel Archive of Georgian History — Artifacts, Monuments & Events

Student: Rezo Darsavelidze

Table of Contents

1. Abstract

GeoArchive is a fully functional, server-rendered CRUD web application built with PHP and the Laravel framework. It preserves and presents Georgian historical artifacts, monuments, manuscripts, and events, allowing the public to browse a curated archive while administrators manage all content through protected workflows.

The project was designed to demonstrate, in one coherent application, the complete set of concepts covered during the course: the Model–View–Controller pattern, database migrations, Eloquent ORM relationships (one-to-one, one-to-many, and two many-to-many relationships), full CRUD with CSRF protection, the Blade templating engine, form validation, authentication, middleware-based access control, file uploads, and a basic JSON API exposed through resource controllers. The interface is written in plain HTML and CSS, using no external front-end framework, so the work that is graded relies strictly on the requested PHP/Laravel technologies.

2. Introduction

The goal of this project is to build a complete database-driven web application that exercises the practical, architectural, and functional areas of Laravel. The chosen theme is a **historical archive of Georgia**: a catalogue of artifacts (objects, monuments, manuscripts) and a chronological collection of historical events, linked together so that visitors can follow how objects relate to the moments in history they belong to.

The application has three audiences:

- **Visitors** browse the public archive: artifacts with search, category and tag filters, sortable chronological events, category and tag pages, and connected-history views.
- **Registered users** manage their own profile, including a biography and an avatar upload.
- **Administrators** use a protected dashboard and full content-management tools for artifacts, categories, tags, and events.

Every record is seeded with genuine, reference-checked historical content and a locally stored, individually sourced image, so the application is meaningful to demonstrate rather than filled with placeholder text.

3. Technologies Used

Area	Technology
Language	PHP 8.2+ (developed and tested on PHP 8.4)
Framework	Laravel 12
ORM	Eloquent
Templating	Blade (server-side rendering)
Database	MySQL / MariaDB (via XAMPP); application database `geoarchive`, tests use `geoarchive_test`
Front-end	Plain HTML and CSS (`public/css/app.css`) — no React, Vue, Tailwind, Bootstrap, or Livewire
Testing	PHPUnit through Laravel's test runner
Authentication	Custom session-based authentication (no external package)

The application requires no Node.js build step, because the final stylesheet is served directly from `public/css/app.css`, and it makes no external API calls at runtime.

4. System Architecture (MVC)

GeoArchive follows the Model–View–Controller pattern strictly, and adds a thin service layer to keep controllers small and to centralize write operations.

Layer	Responsibility	Examples
Model	Eloquent models hold relationships, casts, and reusable query scopes.	`Artifact`, `HistoricalEvent`, `Category`, `Tag`, `User`, `Profile`
View	Blade templates render HTML using a shared layout, partials, and components.	`layouts/app.blade.php`, `artifacts/_card.blade.php`, `components/long-form.blade.php`
Controller	Public and admin controllers handle requests and delegate work; they contain no SQL.	`PublicArchive\ArtifactController`, `Admin\ArtifactController`
Service	Manager classes perform transactional writes and image lifecycle handling.	`ArtifactManager`, `PublicImageStorage`
Request	Form Request classes authorize and validate input away from the controller.	`ArtifactRequest`, `RegisterRequest`
Policy	Per-model authorization rules.	`ArtifactPolicy`, `CategoryPolicy`

A typical write request flows as follows:

```
Request  →  Route (web.php)  →  Middleware (auth, admin)
        →  Controller  →  Form Request (validate + authorize)
        →  Service / Manager  (DB transaction + file storage)
        →  Eloquent Model  →  Database
        →  Redirect with flash message  →  Blade View
```

This separation means each controller action is only a few lines long: it authorizes, calls a manager, and returns a redirect or a view.

5. Database Design & Eloquent Relationships

The schema is defined entirely through Laravel migrations, including foreign keys and two pivot tables. The design covers all three relationship types required by the course.

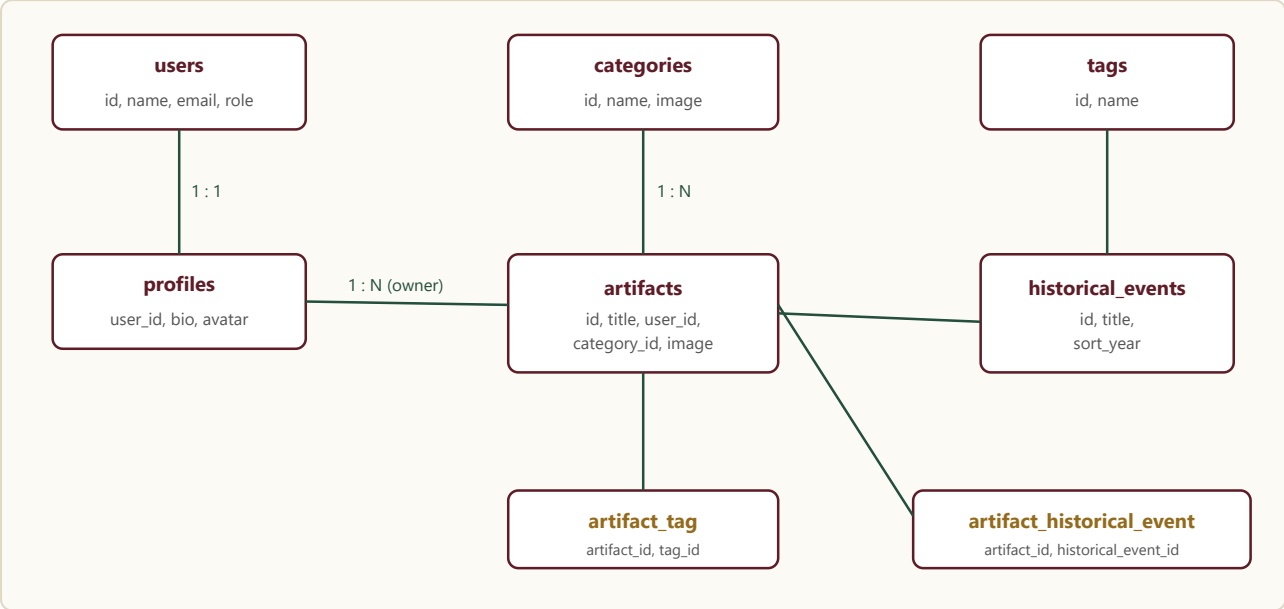

Figure 1 — Entity-relationship overview. Pivot tables (gold) implement the many-to-many relationships.

5.1 Relationship summary

Type	Relationship	Eloquent definition
One-to-One	A user has one profile	`User::profile()` → `hasOne(Profile)`
One-to-Many	A user owns many artifacts	`User::artifacts()` → `hasMany(Artifact)`
One-to-Many	A category has many artifacts	`Category::artifacts()` → `hasMany(Artifact)`
Many-to-Many	Artifacts ↔ tags	`Artifact::tags()` → `belongsToMany(Tag)`
Many-to-Many	Artifacts ↔ historical events	`Artifact::historicalEvents()` → `belongsToMany(HistoricalEvent)`

Foreign keys enforce referential integrity. Deleting a user cascades to that user's profile and artifacts; pivot rows are removed automatically when either side is deleted; and a category that still contains artifacts is protected from deletion both at the database level and with a friendly message in the controller.

5.2 Migrations

Each table, foreign key, and pivot is created by a dedicated migration in `database/migrations`, and an additional migration strengthens indexes used by search and filtering. The schema can be rebuilt at any time with `php artisan migrate:fresh --seed`.

6. CRUD Operations

Administrators have full Create, Read, Update, and Delete capability over four resources, each registered as a Laravel resource controller:

```
Route::resource('artifacts',  AdminArtifactController::class);
Route::resource('categories', AdminCategoryController::class)->except('show');
Route::resource('events',     AdminHistoricalEventController::class);
Route::resource('tags',       AdminTagController::class)->except('show');
```

Every state-changing form includes the `@csrf` directive, and forms repopulate with `old()` input after a validation failure so the user never loses their work. Creating or editing an artifact lets the administrator choose a category, attach multiple tags *and* connect multiple historical events (both many-to-many relationships are managed through the UI), upload an image, and assign an owner.

Operation	Implementation
Create	Resource `create` / `store`; validated by a Form Request; persisted in a database transaction by a manager service.
Read	Public index/show pages with pagination, search, category and tag filters, and chronological sorting.
Update	Resource `edit` / `update`; old images are deleted from disk when replaced or explicitly removed.
Delete	Resource `destroy`; associated image files and pivot rows are cleaned up; protected for categories in use.

7. Blade Templating & User Interface

The interface is built entirely with the Blade engine and demonstrates the required directives and reuse mechanisms:

- `@extends` / `@section` / `@yield` — a single master layout (`layouts/app.blade.php`) provides the navigation, flash messages, and validation-error area for every page.
- `@include` and partials — reusable fragments such as `artifacts/_card.blade.php`, `events/_card.blade.php`, and shared admin `_form.blade.php` files.
- **Blade components** — `<x-long-form>` renders the structured long-form historical text.
- **Control structures** — `@foreach`, `@if`, `@isset`, `@checked`, and `@selected` drive lists, conditional rendering, and form state.
- **Data & error display** — validation errors and success messages are shown from the shared layout, and pagination links preserve the active query string.

8. Security, Validation, Authentication & Middleware

8.1 Authentication

A custom, session-based authentication flow handles registration, login, and logout. Registration validates input and forces the regular-user role; login regenerates the session; logout invalidates the session and regenerates the CSRF token. Login and registration validation live in dedicated `LoginRequest` and `RegisterRequest` classes.

8.2 Middleware & Authorization

An `AdminMiddleware` guards the entire `/admin` route group: unauthenticated visitors are redirected to the login page, and authenticated non-administrators receive an HTTP **403** response. In addition, every managed model has a dedicated Policy (`ArtifactPolicy`, `CategoryPolicy`, `TagPolicy`, `HistoricalEventPolicy`) checked inside the controllers.

8.3 Validation & CSRF

All input is validated through Form Request classes, which combine an `authorize()` method with `rules()`. Titles are unique, descriptions have minimum lengths, and uploaded images are restricted by MIME type, file size, and maximum dimensions. Every form is protected against cross-site request forgery with the `@csrf` token.

9. File Uploads

Artifact, category, event, and avatar images are stored on Laravel's `public` disk and served through the `storage:link` symlink. Upload handling is centralized in a `PublicImageStorage` service whose `storeSafely()` method is transaction-aware: if the database operation that follows an upload throws an exception, the newly stored file is automatically removed, so the disk never accumulates orphaned images. When an image is replaced or its record deleted, the previous file is removed, and administrators can explicitly clear an existing image without uploading a replacement.

Upload	Storage location
Artifact image	`storage/app/public/artifacts`
Category image	`storage/app/public/categories`
Event image	`storage/app/public/events`
Profile avatar	`storage/app/public/avatars`

10. REST / JSON API

Alongside the server-rendered interface, the application exposes a small read-only JSON API built with core Laravel resource controllers and API Resources. The routes are declared in `routes/api.php`, automatically prefixed with `/api` and assigned the stateless, rate-limited `api` middleware group.

Method & Endpoint	Returns
`GET /api/artifacts`	Paginated JSON collection of artifacts (with `data`, `links`, `meta`); supports `?q=` search and `?category=` filter.
`GET /api/artifacts/{id}`	A single artifact with its category, tags, and connected events.
`GET /api/events`	Chronological JSON collection of historical events.
`GET /api/events/{id}`	A single event with its connected artifacts.

Responses are shaped by `ArtifactResource` and `HistoricalEventResource` (subclasses of `JsonResource`), which expose the many-to-many relationships as readable arrays. Example response for `GET /api/events/1`:


```
{
  "data": {
    "id": 1,
    "title": "Colchis in Western Georgia",
    "date_or_period": "Late Bronze Age–1st century BCE",
    "sort_year": -1200,
    "location": "Western Georgia and the eastern Black Sea coast",
    "connected_artifacts": ["Vani Bronze Figurine", "Vani Colchis Coin"],
    "links": { "web": ".../events/1", "api": ".../api/events/1" }
  }
}
```


Missing records return a proper JSON `404` response, demonstrating correct API error handling.

11. Additional Features

Two browsing features make the many-to-many relationships tangible. Both are implemented with the requested technologies only (Blade plus inline SVG and vanilla JavaScript; no external libraries).

- **History Paths** — curated, linear journeys that walk the visitor through related artifacts and events as one connected story; every stop links to the full record.

- **History Graph** — an interactive, force-directed graph in which every artifact and event is a node, edges follow the chronological timeline and the artifact–event pivot, and tapping a node opens its detail page. This visually demonstrates the `artifact_historical_event` many-to-many relationship.

12. Testing & Quality Assurance

The project ships with an automated test suite executed through `php artisan test`. At the time of writing, **33 tests pass (607 assertions)**, covering both feature and unit levels.

- Registration, login, role-based redirect, and logout.
- Regular-user denial (HTTP 403) and administrator access to admin pages.
- Full CRUD for artifacts, categories, tags, and events, including image upload and relationship syncing.
- Safe category deletion, duplicate-title rejection, and unsafe-upload rejection.
- Public pages, search, filtering, and chronological sorting.
- Repeatable seeding and uniqueness of every seeded image and source record.
- Transaction-safe storage (orphan-file cleanup) and policy decisions.
- JSON API collection, single-item, search, and 404 responses.

13. Installation & Running the Project

```
composer install
cp .env.example .env            # (Windows: Copy-Item .env.example .env)
php artisan key:generate
php artisan migrate:fresh --seed
php artisan storage:link
php artisan serve               # open http://127.0.0.1:8000
```

Demonstration accounts

Role	Email	Password	Lands on
Administrator	`admin@geoarchive.test`	`password`	`/admin/dashboard`
Regular user	`user@geoarchive.test`	`password`	`/`

These credentials are for local demonstration only.

14. Rubric Coverage Mapping

The table below maps each grading block to the concrete evidence in the codebase.

Rubric block (10 pts each)	Where it is demonstrated
MVC & Database Management	Public/Admin controllers + service layer + Blade; all migrations with foreign keys and two pivot tables; one-to-one, one-to-many, and two many-to-many Eloquent relationships.
CRUD Operations	Four resource controllers with full Create/Read/Update/Delete; `@csrf` on every form; `old()` repopulation; transactional managers.
Blade Templating & UI	Master layout with `@extends` / `@section`; `@include` partials; an `<x-long-form>` component; `@foreach` / `@if`; validation-error and flash-message display; query-preserving pagination.
Security, Validation & Advanced Features	Form Request validation + authorization; custom authentication; `AdminMiddleware` with 403; per-model policies; transaction-safe file uploads; resource controllers; and a read-only JSON API with API Resources.

15. Conclusion

GeoArchive is a complete, runnable Laravel application that demonstrates every concept required by the course within a single, coherent, and meaningful domain. It separates responsibilities cleanly across the MVC layers, manages its schema through migrations, models all three Eloquent relationship types (including two distinct many-to-many relationships), provides full CRUD with CSRF protection and validation, renders its interface entirely through Blade, secures administrative areas with authentication and middleware, handles file uploads safely, and exposes a basic JSON API through resource controllers. Its automated test suite confirms that these behaviours work as intended.

16. References

- Laravel 12 Documentation — `https://laravel.com/docs`
- Eloquent ORM, Migrations, Validation, Blade, and Eloquent API Resources (official Laravel guides).
- Historical content cross-checked against public encyclopaedic summaries (e.g. Colchis, the Kingdom of Iberia, the Battle of Didgori, the Treaty of Georgievsk, the 1991 independence referendum).

- Seed images are individually sourced from Wikimedia Commons and documented in `IMAGE_CREDITS.md` / `IMAGE_SOURCES.json`.